

Informe Técnico Analítico

-Proyecto Práctico Integrador-

Asignatura: Paradigmas y Lenguajes

Tema: Sistema de Semáforos Inteligente

Grupo 5:

- Aguirre Valentín Emmanuel
- Aquino Luciano Ramiro
- Chavez Tobías Acdel
- Espinoza Martín Ariel
- Rodas Juan Martín

ÍNDICE

Tabla de contenido

Diseño funcional adoptado	3
Fase 1	3
Requerimiento 1	3
Requerimiento 2	4
Requerimiento 3	4
Requerimiento 4	4
Requerimiento 5	5
Requerimiento 6	5
Requerimiento 7	5
Fase 2	5
Bitácora de Depuración	6
 Investigación de la Fase 3	 10
Descripción	10
Historia	10
Características Principales	10
Paradigmas Funcional	11
Inmutabilidad	13
Pureza Funcional	13
Ventajas	14
Fundamentos Teóricos	15
Tipado Dinámico vs. Tipado Estático	15
Algebraic Data Types (ADT)	15
¿Cómo el compilador evita estados inválidos?	16
Lazy Evaluation	16
Programación en Haskell	17
Aplicaciones Reales	17
Empresas que utilizan Haskell	18
 Conclusión Grupal	 20
Bibliografía	21

Diseño funcional adoptado

Fase 1

Tras la primera lectura de las consignas, nos repartimos entre el grupo los requisitos y nos hicimos varias preguntas: ¿Incluimos una función principal? ¿Cómo realizamos el pase por parámetros? ¿Y las validaciones?

Cabe resaltar, que evaluamos el ciclo que debía tener el semáforo y lo decidimos como un ciclo de tipo “rojo-amarillo-verde-amarillo” observando los semáforos que tenemos acá. De la misma forma, decidimos implementar los cambios de la segunda iteración y la fase 2 sobre el mismo código, debido a que GitHub guarda las fases anteriores del código y se puede revisar la fase 1 en cualquier momento.

Entonces, mientras cada uno realizaba su requerimiento, íbamos comparando y produciendo los casos de prueba que dejamos comentado en el código. En un inicio creamos una función llamada “sistema-semáforo” con la intención de llamar a las demás funciones y pasarle los parámetros ingresados por operador, sin embargo, sobre el final decidimos eliminarla porque pocas funciones compartían parámetros, además, consideramos que muchas funciones tienen un uso que se adecua a las necesidades del cliente, por lo que no podíamos pensar una función principal que llame un número finito de veces a funciones que tienen más utilidad al llamarse varias veces durante el programa.

Para los parámetros y validaciones quedó a cargo de cada uno validar su función, mientras que los parámetros compartimos la misma denominación para mantener una consistencia, siendo: “segundos” para el tiempo ingresado, “estadoActual” para el color que tiene el semáforo en el momento y “cambiar” para el siguiente color. Los símbolos permitidos son “en-rojo”, “en-verde”, “en-amarillo” e “intermitente”.

REQUERIMIENTO 1

Creamos la función “transición” que recibe el color actual y al que cambiará para crear devolver una lista con el actual y el cambio a realizar. Esto lo implementamos mediante una estructura cond que verifica los símbolos recibidos para ver si pertenecen a alguna de las opciones, de lo contrario devuelve una acción por defecto. Por ejemplo, si ingresamos “en-rojo” y “amarillo”, en la estructura cond se verificará que el primer símbolo sea igual a alguno de los permitidos, lo mismo que el segundo símbolo y, si ambos son iguales, creará la lista “(EN-ROJO cambiar a verde)”.

Luego, modificamos la función para cumplir con lo requerido por la extensión 1 de la iteración dos. Esto lo hicimos implementando un nuevo símbolo, “intermitente” que se ubica entre los colores, agregamos 3 opciones más a la estructura cond con el nuevo símbolo.

REQUERIMIENTO 2

Elaboramos la función “timer” que recibe como parámetro “segundos” y tras una operación devuelve el color al que pertenece el tiempo indicado por el parámetro. Esto lo realizamos mediante una variable local auxiliar “tiempo-ciclo” que calcula el resto de la división entre el tiempo ingresado y 216 (cantidad de segundos en un ciclo) y mediante un cond verifica el rango al que pertenece tiempo-ciclo. Devuelve ROJO al estar entre 0 y 90, AMARILLO al estar entre 90 y 96 y VERDE al estar entre 96 y 216.

Para la extensión 1, modificamos el valor de 216 por 225, que es el fruto de sumar el 216 con los 9 segundos que agregan las luces intermitentes (3 segundos cada una). Luego, agregamos la opción de devolver “INTERMITENTE” como estado actual si se encuentra en alguno de los rangos de entre 90 y 93, entre 99 y 102, y entre 222 y 225. Como medida ante algún error, si de alguna forma el resultado es menor a 0 o mayor a 225, la función devolverá ERROR.

REQUERIMIENTO 3

Hicimos la función “logging-auditoria” que recibe los tres parámetros iniciales e imprime un mensaje en pantalla indicando el tiempo, el color anterior y a qué color cambió. Esto lo elaboramos mediante una función con “format t” para que imprima los parámetros en un formato adecuado. Format t por naturaleza devuelve el último valor, y como en nuestra función solo imprimía en pantalla, devolvía NIL. Para arreglar esto y hacerlo más legible, pusimos la línea “REGISTRADO-EXITOSAMENTE” para que la devuelva luego de imprimir la línea.

Modificamos la función para la extensión 2, ahora la función guarda las líneas en un archivo en lugar de imprimirlas en pantalla. Logramos esto con la función with-open-file, que verifica si existe un archivo, si no existe lo crea y si existe ejecuta las instrucciones y cierra el archivo automáticamente.

Cabe destacar, que queríamos que el archivo fuese más legible mediante un encabezado antes de las líneas que iba escribiendo el programa, esto lo logramos con la función auxiliar “crea-informe” la cual va a llamarse una única vez para crear el informe y escribir el encabezado, entonces logging-auditoria solo abre el archivo ya creado y escribe una línea al final, sin sobrescribir el encabezado ni volver a escribirlo.

REQUERIMIENTO 4

Creadas las funciones “duración-ciclo” y “recomendación-ciclo”. La primera recibe las duraciones de cada color y, mediante un if, valida que sean enteros para luego sumarlas y calcular la duración total de un ciclo. La segunda función toma el valor de duración calculado por la primera y con una estructura cond verifica si la duración está en el rango recomendado (35-150 segundos) y devuelve un mensaje.

Para la extensión 1, solo se modificó la función duración-ciclo para agregarle el tiempo de las luces intermitentes al cálculo de la duración total.

REQUERIMIENTO 5

La función “ciclos-por-tiempo” recibe una cantidad determinada de minutos y calcula cuántos ciclos se cumplen en ese tiempo. Esto lo hicimos al multiplicar los minutos por 60 para obtener la cantidad de segundos y luego dividirlo por 216 que es la duración del ciclo básico. Utilizamos la función floor para redondear y que no queden decimales.

Para la extensión 1 solo se modificó el valor 216 por 225.

REQUERIMIENTO 6

Realizamos la función “informe-distribucion-60min” que calcula la distribución porcentual de cada color que se tendrá en períodos de 1 hora. Para que esta función funcione diseñamos la función auxiliar “calcular-porcentaje” que divide el tiempo total del color (90 para rojo, 6 para amarillo, 120 para verde) por la duración del ciclo, multiplicamos el resultado por los segundos en una hora y lo dividimos por 100 para obtener el porcentaje.

En “informe-distribucion-60min” se crea una lista de sublistas, con las sublistas siendo un color y el porcentaje, calculado al llamar a la función “calcular-porcentaje”.

Para la extensión 1, se agrego el porcentaje de INTERMITENTE y se le sumó 9 segundos a la duración del ciclo.

REQUERIMIENTO 7

Agregamos casos de prueba, alternativos y de error como comentarios luego de cada función en el código.

Fase 2

Una vez finalizada la fase 1, pasamos a la pregunta de qué librería implementar. Decidimos entonces implementar la librería local-time por su uso y por lo legible que se veían las líneas en el informe con esa librería.

Investigamos entonces como instalar quicklisp y utilizar la librería, para ello descargamos desde la página oficial quicklisp, lo instalamos en el clisp de SublimeText y cargamos la librería.

Modificamos la función “logging-auditoria” para que use la librería e imprima con un formato más legible. Para ello utilizamos la variable local auxiliar fechalnicio a la que le guardamos una fecha base desde la que se le sumaría el tiempo ingresado como parámetro para imprimir la fecha del cambio en el archivo. Establecimos un formato “año:mes:día:hora:minuto:segundo:nanosegundo” mediante local-time:format-timestring.

La función final entonces recibe una cantidad de segundos, el color actual y al que cambiará por parámetros, entonces suma los segundos a la fecha base e imprime en el archivo la línea con la fecha legible. La función crea-informe se mantuvo intacta.

Bitácora de Depuración

Durante el desarrollo del sistema de semáforos inteligentes surgieron diversos errores relacionados con la sintaxis, validación de parámetros, diseño de funciones, condiciones mal acomodadas o mal planteadas, a continuación se hace la muestra de estos errores y algunos más:

- **Problema con comillas en función Timer:** En la última validación del cond, como caso de error se hace la impresión del mensaje ERROR, pero se cometió el descuido de confundir el orden de la comilla para la impresión de dicho mensaje, la solución fue solo cambiar de lugar la misma comilla al comienzo del mensaje.

```

53 53 ((and (>= tiempo-ciclo 0) (<= tiempo-ciclo 89))
54 54 'ROJO)
55 55 ((and (>= tiempo-ciclo 90) (<= tiempo-ciclo 95))
56 56 'AMARILLO)
57 57 ((and (>= tiempo-ciclo 96) (<= tiempo-ciclo
58 58 215))) 'VERDE)
59 59 (t 'ERROR')

```

- **Condición mal implementada en recomendación-ciclo:** En la función recomendación-ciclo nos dimos cuenta que la condición de mayor o igual estaba repetida y no devolvía los resultados esperados, su corrección fue que se la paso a menor o igual.

```

143 143 (defun recomendacion-ciclo(total-ciclo)
144 144 (cond
145 145 ((> 35 total-ciclo) "recomendacion: aumentar el tiempo del ciclo
146 146 para obtener entre 35 a 150 segundos")
147 147 ((and (>= total-ciclo 35) (<= 150 total-ciclo)) "¡su tiempo esta
148 148 en los estandares optimos!")
149 149 ((and (>= total-ciclo 35) (< 150 total-ciclo)) "¡su tiempo esta
150 150 en los estandares optimos!")
151 151 ((> total-ciclo 150) "recomendacion: disminuya su tiempo paraa
152 152 obtener un ciclo entre 35 a 150 segundos")

```

- **Paréntesis de más en informe-distribucion-60min:** Tratando de compilar la función informe-distribucion-60min nos dimos cuenta que por error de tipeo se agregó un paréntesis de más en la línea 223, su corrección fue quitar ese paréntesis extra.

```

218 218 (defun informe-distribucion-60min ()
219 - (let ((ciclo-total (+ 90 6 120))) ; Duración de un ciclo completo
    (216s)
220 - (list
221 - (list 'ROJO (calcular-porcentaje 90 ciclo-total))
222 - (list 'AMARILLO (calcular-porcentaje 6 ciclo-total))
223 - (list 'VERDE (calcular-porcentaje 120 ciclo-total)))))
219 + (let ((ciclo-total (+ 90 6 120))) ; Duración de un ciclo completo
    (216s)
220 + (list
221 + (list 'ROJO (calcular-porcentaje 90 ciclo-total))
222 + (list 'AMARILLO (calcular-porcentaje 6 ciclo-total))
223 + (list 'VERDE (calcular-porcentaje 120 ciclo-total))
224 + )
225 + )
226 + )
227 +
228 + ;; Caso normal
229 + (informe-distribucion-60min)

```

- **Eliminación de let* en función informe-distribucion-60min:** primero definimos la función de esta forma implementando una estructura de ligadura secuencial let* para calcular y almacenar localmente los identificadores tiempo-total (3600) y proporción ((/ tiempo-total ciclo-total)) pero al final estas variables jamás fueron invocadas ni utilizadas en la construcción de la lista de salida final. Entonces se eliminó el bloque let* innecesario y las variables redundantes. Se reestructuró la solución de manera modular mediante una función aritmética directa (calcular-porcentaje), aplicando un let simple únicamente para la constante indispensable ciclo-total. Esto redujo el uso de memoria y mejoró la legibilidad del código.

```

(defun informe-distribucion-60min ()
  (let* ((tiempo-total 3600)
        (ciclo-total (+ 90 6 120)) ; Duración de un ciclo completo (216s)
        (proporcion (/ tiempo-total ciclo-total)) ; Cuántas veces entra el ciclo en 1h
        (list
          (list 'ROJO (calcular-porcentaje 90 ciclo-total))
          (list 'AMARILLO (calcular-porcentaje 6 ciclo-total))
          (list 'VERDE (calcular-porcentaje 120 ciclo-total)))))

```

```

(defun informe-distribucion-60min ()
  (let ((ciclo-total (+ 90 6 120))) |
    (list 'ROJO (calcular-porcentaje 90 ciclo-total))
    (list 'AMARILLO (calcular-porcentaje 6 ciclo-total))
    (list 'VERDE (calcular-porcentaje 120 ciclo-total)))

```

- Eliminación de let innecesario en calcular-porcentaje:

```
(defun calcular-porcentaje (tiempo-color tiempo-ciclo)
  (let ((resultado (* (/ tiempo-color (float tiempo-ciclo)) 100)))
    (/ (round (* resultado 100)) 100.0)))
```

```
(defun calcular-porcentaje (tiempo-color ciclo-total)
  (/ (round (* (* (/ tiempo-color (float ciclo-total)) 100) 100)) 100.0))
```

- Mala estructura en función duración-ciclo: reestructuración de un let simple que abarcaba parte del código donde no era usado y eliminación de un return innecesario.

```
(defun duracion-ciclo(rojo amarillo verde)
  (let ((tiempo-un-ciclo (+ rojo amarillo verde)) )
    (cond
      (
        (not (and (integerp rojo) (integerp amarillo) (integerp verde)))
        "error: uno de los parametros no es numero entero"
      )
      (
        (format t "el ciclo dura ~a's" tiempo-un-ciclo)
        (return tiempo-un-ciclo)
      )
    )
  )
)
```

```
(defun duracion-ciclo(rojo amarillo verde)
  (cond
    (
      (not (and (integerp rojo)(integerp amarillo)(integerp verde)) )
      "Error: uno de los parametros no es un numero entero"
    )
    (t
      (let ((tiempo-un-ciclo (+ rojo amarillo verde)) )
        (format t "El ciclo dura ~a's~%" tiempo-un-ciclo)
        tiempo-un-ciclo
      )
    )
  )
)
```

- Reestructuración de la función logging-auditoria: la función imprimía nuevamente el título cada vez que se llamaba. Se solucionó con la creación de una función auxiliar que crea el informe y escribe una única vez el título del informe.

```
Break 2 [7]> (defun logging-auditoria (segundos estadoActual cambiar)
  (with-open-file (stream "informe-ejecucion-semaforo.txt" :direction :output :if-exists :append)

    (format stream "Informe de Ejecución del Sistema Semafórico~%")
    (format stream "=====~%")

    (format stream
      "~%Tiempo ~a: la luz ha cambiado de ~a a ~a~%" segundos estadoActual cambiar)
  ))
```



```
Informe de Ejecución del Sistema Semafórico
=====

Tiempo 100: la luz ha cambiado de EN-ROJO a VERDE

--- Fin del Informe ---Informe de Ejecución del Sistema Semafórico
=====

Tiempo 100: la luz ha cambiado de EN-ROJO a VERDE
Informe de Ejecución del Sistema Semafórico
=====

Tiempo 5000: la luz ha cambiado de EN-ROJO a VERDE
Informe de Ejecución del Sistema Semafórico
=====

Tiempo 3500: la luz ha cambiado de EN-VERDE a AMARILLO
```

Investigación de la Fase 3

Haskell

Lenguaje funcional puro, con tipado estático fuerte y evaluación perezosa. Diseñado por la comunidad académica para unificar la investigación en programación funcional.

Descripción

Historia

- 1987
- En la conferencia FPCA, investigadores acuerdan crear un lenguaje funcional puro estándar. Existían más de una docena de lenguajes similares y la fragmentación era un problema.
- 1990
- **Haskell 1.0** – primera especificación oficial, nombrado en honor al lógico Haskell Curry, pionero del cálculo lambda y la lógica combinatoria.
- 1992-1997
- Versiones 1.1 a 1.4 refinan el sistema de tipos, las clases de tipos y la sintaxis. El compilador GHC (Glasgow Haskell Compiler) se vuelve la implementación de referencia.
- 1999
- **Haskell G8** – estándar estable pensado para enseñanza e investigación. Define el núcleo del lenguaje que persiste hasta hoy.
- 2010
- **Haskell 2010** – última revisión oficial. Agrega importaciones jerárquicas y mejoras de compatibilidad. GHC continúa añadiendo extensiones del lenguaje activamente.
- 2015-HOY
- Adopción creciente en finanzas (Standard Chartered, Facebook), compiladores (GHC mismo), y herramientas de análisis. Stack y Cabal estandarizan la gestión de proyectos.

Características Principales

- I. **Tipado estático fuerte:**
Los tipos se verifican en tiempo de compilación. Si compila, es muy probable que funcione correctamente.

II. Inferencia de tipos:

El compilador deduce los tipos automáticamente. Raramente es necesario anotarlos de forma explícita.

III. Evaluación perezosa:

Las expresiones se evalúan solo cuando su valor es necesario. Permite trabajar con estructuras infinitas.

IV. Funciones de primera clase:

Las funciones son valores: se pasan como argumentos, se retornan y se componen libremente.

V. Clases de tipos:

Sistema de polimorfismo ad-hoc más poderoso que las interfaces OOP. Fundamenta abstracciones como Functor y Monad.

VI. Pattern matching:

Deconstrucción elegante de datos. Reemplaza cascadas de if/else con código declarativo y exhaustivo.

Paradigmas Funcional

En Haskell, un programa es una transformación de datos expresada como la composición de funciones matemáticas. No hay instrucciones imperativas: no hay bucles for, no hay variables que muten, no hay estado global.

Ejemplo 1: Imperativo vs Funcional

-- Imperativo (pseudocódigo)

suma = 0

for i in [1..10]:

suma += i

-- Funcional en Haskell

suma = sum [1..10] ---> 55

Recursión en lugar de bucles

La iteración se expresa mediante funciones recursivas o combinadores como map, filter y fold.

Composición de funciones

Funciones pequeñas se combinan con el operador (.) para construir transformaciones complejas.

Funciones de orden superior

map, filter y foldr abstraen los patrones de iteración más comunes.

Ejemplo 2: Composición de funciones

-- Sin composición (anidado)

*resultado xs = sum (filter even (map (*2) xs))*

-- Con composición: se lee de derecha a izquierda

*resultado = sum . filter even . map (*2)*

-- Uso:

resultado [1..10]

- Paso 1: *map (*2)-----> [2,4,C,8,10,12,14,1C,18,20]*

- Paso 2: *filter even -----> [2,4,C,8,10,12,14,1C,18,20]*

- Paso 3: *sum-----> 110*

Ejemplo 3: map, filter foldr

-- map: doblar cada número

*map (*2) [1,2,3,4,5]----> [2,4,C,8,10]*

-- filter: conservar solo pares

filter even [1..10]----> [2,4,C,8,10]

-- foldr: sumar todos los elementos

foldr (+) 0 [1..10]---> 55

Inmutabilidad

En Haskell, todos los valores son inmutables. No existe forma de cambiar el valor de una variable una vez asignada. En lugar de mutar, se crean nuevas versiones de las estructuras.

-- Esto NO existe en Haskell: **let x = 5 x = 6** -- Error de compilación

-- En cambio, se crean nuevos valores: **let x = 5 let x' = x + 1 --- x' = 6**, x sigue siendo 5

Sin efectos laterales locales

Una función no puede modificar datos fuera de su scope. El resultado depende únicamente de sus argumentos.

Seguridad en concurrencia

Sin estado mutable compartido, las condiciones de carrera (race conditions) son eliminadas estructuralmente.

Razonamiento temporal

Un valor que es 5 al principio del programa seguirá siendo 5 al final.

-- En nuestro sistema, cada llamada a **transicion-ext-1**

-- devuelve un nuevo estado sin modificar el original

(transicion-ext-1 'en-rojo 'intermitente)

;; Resultado: (EN-ROJO "cambiar a intermitente")

;; 'en-rojo no fue modificado, se devolvió una nueva lista

Pureza Funcional

Una función pura produce siempre el mismo resultado para los mismos argumentos y no tiene efectos observables en el exterior (sin I/O, sin excepciones silenciosas, sin estado global). Haskell garantiza esta propiedad a nivel de tipos.

-- Función pura: mismo input ---> mismo output, siempre

(timer-ext-1 35) ;; → AMARILLO (siempre)

(timer-ext-1 150) ;; → VERDE (siempre)

Testeabilidad perfecta:

Las funciones puras no necesitan mocks, stubs ni bases de datos de prueba. Una lista de casos es suficiente.

Mónadas para efectos:

Los efectos impuros (I/O, estado, errores) se modelan con mónadas, aislándolos del código puro de forma tipada.

Transparencia referencial

Cualquier expresión puede reemplazarse por su valor sin cambiar el comportamiento del programa.

Ventajas

1. **El compilador como red de seguridad:** El sistema de tipos captura en tiempo de compilación clases enteras de bugs que en otros lenguajes solo aparecen en producción.
2. **Concurrencia sin dolor:** La inmutabilidad elimina las condiciones de carrera. Las primitivas STM (Software Transactional Memory) hacen el paralelismo composable.
3. **Código más corto y expresivo:** La composición de funciones y las abstracciones de alto nivel permiten expresar lógica compleja en pocas líneas sin sacrificar claridad.
4. **Refactoring sin miedo:** Cambiar una función pura no produce efectos inesperados en otras partes del sistema. El compilador detecta todas las inconsistencias.
5. **Mejora tu pensamiento como programador:** Aprender Haskell transforma la manera de modelar problemas en cualquier lenguaje, incluyendo Python, JavaScript o Rust.
6. **Performance sorprendente:** GHC genera código nativo altamente optimizado. La evaluación perezosa y la fusión de listas evitan copias intermedias en memoria.

Fundamentos Teóricos

Tipado Dinámico vs. Tipado Estático

Para entender la diferencia, hay que analizar cuándo el lenguaje sabe el tipo de dato que está procesando:

LISP (Tipado Dinámico): Los tipos de datos están vinculados a los valores, no a las variables o funciones. El compilador permite que el código sea flexible y "acepte cualquier cosa" durante la escritura. El chequeo se hace en tiempo de ejecución (runtime). Por ejemplo: Si en la función `timer-ext-1` se coloca un string en lugar de los segundos, el sistema falla recién cuando se ejecuta esa línea.

HASKELL (Tipado Estático): Los tipos están vinculados a las expresiones y funciones en tiempo de compilación. Por ejemplo, la firma `timer :: Int -> String` es inquebrantable. El compilador de Haskell realiza un proceso llamado Inferencia de Tipos (Algoritmo Hindley-Milner) analizando todo el árbol del código antes de generar el archivo ejecutable. Si detecta una inconsistencia, el programa directamente no compila.

Impacto en el Semáforo: El tipado estático elimina el 100% de los errores por parámetros inválidos. No hace falta programar funciones defensivas con `if (numberp ...)` como en Lisp, el compilador garantiza que a la función `timer` solo van a entrar números enteros (`Int`).

Algebraic Data Types (ADT) o Tipos de Datos Algebraicos

En Lisp, para representar estados como "Rojo", "Amarillo" o "Verde", se suele recurrir a strings o símbolos sueltos. Esto puede llevar a cometer errores como un simple error de tipeo como `"rojo "` con un espacio extra que rompe la lógica.

Haskell resuelve esto mediante los Algebraic Data Types (ADT). Se denominan "algebraicos" porque se forman combinando otros tipos mediante dos operaciones principales:

Tipos Producto: Combinan múltiples valores (equivalente a un registro o estructura).

Tipos Suma: Representan una elección entre un conjunto de alternativas disyuntas (utilizando el operador `|`).

Para nuestro sistema de semáforos lo ideal sería del tipo suma

```
data EstadoSemaforo = Rojo | Amarillo | Verde
```

Rojo, Amarillo y Verde no son strings ni enteros; son constructores de tipo puros. El tipo `EstadoSemaforo` contiene únicamente la suma de esas tres posibilidades históricas. No existe ninguna otra opción en su universo.

¿Cómo el Compilador evita Estados Inválidos?

El compilador de Haskell utiliza los ADT en combinación con el Pattern Matching (Coincidencia de Patrones) para blindar la lógica del negocio.

Para la función “transicion :: String -> String -> String” el compilador se ve obligado a aceptar cualquier cadena de texto (como "hola" o "azul"), teniendo que delegar la seguridad al caso por defecto “otherwise = "accion por defecto" ”.

El Compilador de Haskell (GHC) realiza un Análisis Exhaustivo (Exhaustiveness Checking), por ejemplo, si no se qué pasa cuando el semáforo está en Amarillo, el compilador frena la compilación con un error de "Non-exhaustive patterns", lo que obliga a cubrir todas las combinaciones posibles del mundo real. Además, es imposible pasarle al sistema un estado que no exista. El compilador rechaza cualquier valor que no sea uno de los constructores definidos en el ADT. El software se vuelve matemáticamente consistente y los estados inválidos quedan físicamente fuera del programa.

Lazy Evaluation (Evaluación Perezosa) y Flujos Infinitos

La mayoría de los lenguajes utilizan Evaluación Estricta, donde los argumentos de una función se calculan por completo antes de ejecutar la función misma.

Haskell utiliza Evaluación Perezosa (Lazy Evaluation). Bajo este mecanismo, una expresión no se evalúa (no se procesa en el procesador ni ocupa espacio real en la memoria) hasta que su resultado sea estrictamente necesario para una operación de salida (como imprimir en pantalla o guardar en un archivo). El compilador maneja esto internamente mediante unas estructuras llamadas thunks (punteros de expresiones suspendidas).

Gracias a la evaluación perezosa de Haskell, se pueden modelar conceptos matemáticos continuos o infinitos. En un semáforo real, el tiempo no se frena a la hora; el tiempo es un flujo continuo. Si tuviéramos un flujo infinito de eventos de tráfico (por ejemplo, un sensor en la calle mandando datos de segundos sin parar: [1, 2, 3, 4, ...]), en cualquier otro lenguaje el sistema colapsaría por falta de memoria RAM al intentar almacenar o procesar el infinito, pero en Haskell la lista infinita de eventos no se calcula por adelantado. Se crea un thunk (una promesa de cálculo). Si el controlador del semáforo real solo necesita saber el estado de los próximos 3 segundos, se ejecuta un take 3 sobre el flujo infinito. Se calcula esos 3 segundos usando el timer y el resto de la lista infinita queda suspendido en memoria sin consumir recursos.

Programación en Haskell

Reflexión sobre la experiencia con Haskell

La experiencia de trabajar con Haskell fue interesante y desafiante al mismo tiempo. Al principio resultó complicado porque era un lenguaje completamente nuevo para mí y tuve que aprender su sintaxis y forma de pensar desde cero. Una de las primeras dificultades fue la instalación y configuración del entorno, ya que tuve varios problemas para lograr que el compilador y GHCi funcionaran correctamente en mi computadora.

Una vez superada esa etapa, el proceso de aprendizaje se volvió más entretenido. Me llamó la atención la diferencia con otros lenguajes que había utilizado anteriormente, especialmente por su tipado estático y su enfoque funcional. Durante el desarrollo también aparecieron errores relacionados con la sintaxis, la carga de archivos en GHCi y el uso de variables y funciones, pero resolverlos me ayudó a comprender mejor cómo funciona el lenguaje.

En general, considero que fue una experiencia positiva porque me permitió salir de mi zona de confort y aprender un paradigma de programación diferente por mis propios medios. Aunque al principio fue frustrante, resultó satisfactorio lograr implementar las funciones solicitadas y entender conceptos nuevos que antes no conocía.

Aplicaciones Reales

Se utiliza con más normalidad en los sectores financieros llegando a niveles industriales de estos mismos.

mas en la actualidad hablamos de sistemas de trading, sistemas de analisis riesgo, en motores para contratos financieros,

en procesosos de transacciones ya sean bancos o infraestructuras de blockchain q se ocupan en criptomonedas y micro transacciones en este.

Empresas destacadas que utilizan este lenguaje:

-Standar Chárted: Este desarrollo un sistema en haskell la cual la utiliza en una plataforma financiera llamada MU, principalmente utiliza este para modelar productos financieros complejos.

-Barclays: esta plataforma se centra principalmente trading el cual eligió incorporar haskell para herramientas internas para el análisis de los valores y directamente con el trading

-IOG: es una empresa basada principalmente en transacciones en un blockchain q da espacio para q se realicen estas de manera eficiente sin q los usuarios deban tener esta infraestructura, en este caso eligieron usar haskell para las bases de la logistica y rendimiento del sistema.

ACTUALIDAD

Como vimos en los ejemplos lo más usual es usar este lenguaje tan potente para servidores de alta concurrencia, sistemas de mensajería, infraestructura en la nube, utilización de APIs.

Uno de los proyectos más destacados es que META, empezó utilizando haskell aprovechando la capacidad de ejecutar estructuras complejas y con concurrencias utilizado en: Generadores de códigos, Analizadores sintéticos y verificadores formales.

Empresas que utilizan Haskell

El lenguaje Haskell se usa principalmente para:

- Finanzas y banca.
- Criptomonedas y blockchain.
- Compiladores y herramientas de programación.
- Sistemas donde los errores son muy costosos.
- Investigación y universidades.

Teniendo bastante uso comercial, desde aeroespacial y defensa, financiero, diseño de hardware y hasta una empresa que diseña cortadoras de césped. Existe un repositorio en GitHub con información de empresas que actualmente usan Haskell, de donde sacamos gran parte de esta información.

Algunas empresas importantes que usaron o usan Haskell:

- **Alcatel-Lucent:** marca de telecomunicaciones, productoras de celulares, telefonía de consumo y servidores de comunicación. Ha usado Haskell para un proyecto de programación en tiempo real para comprobar si el lenguaje se podía usar en un proyecto práctico con restricciones temporales exigentes. Como beneficios

podieron sacar que Haskell desarrolló más rápido en menos código, mayor detección de errores antes de ejecutar el código gracias al sistema de tipos que tiene el lenguaje y el código fue más fácil de entender al ser funciones puras.

- **Facebook (Meta):** aunque para casi todo el desarrollo se usó PHP, algunos equipos usaron Haskell para resolver problemas complejos de forma rápida, como por ejemplo detección de spam, análisis de grandes volúmenes de datos, y prototipado rápido de algoritmos. Se beneficiaron del uso de este lenguaje en que podían realizar funciones complejas con menos código, era más sencillo cambiar partes del código por sus abstracciones (necesario al tener que cambiar constantemente el sistema de detección de spam) y se presentaron menos errores al ser más fácil ver cuando algo salía mal.
- **Google:** utilizó Haskell en un proyecto que administraba infraestructuras de máquinas virtuales, el cuál era programado en Python pero había problemas que Python no podía expresar de forma segura por lo que desarrollaron algunas partes en Haskell donde se necesitaba corrección y modelado de datos. Obtuvieron beneficios en una mejor comprensión de los algoritmos, interfaces más limpias debido a lo estricto del lenguaje en cuanto a los tipos y la conclusión de que Haskell era un buen complemento para los problemas que Python no podía resolver de forma eficiente.

Conclusión Grupal

A lo largo del trabajo fuimos viendo Haskell desde distintos lados: de dónde viene, cómo funciona, dónde se usa y finalmente lo pusimos en práctica con el sistema de semáforos.

Lo más interesante fue darnos cuenta de que la forma en que se programa en Haskell te obliga a pensar distinto. En vez de decirle a la computadora paso a paso qué hacer, describís el resultado que querés y el lenguaje se encarga del resto. Al principio cuesta acostumbrarse, pero una vez que le agarrás la mano el código queda mucho más limpio y fácil de entender.

También notamos que dividir bien las funciones, las que solo calculan cosas de las que guardan archivos o muestran por pantalla, hizo que sea mucho más fácil encontrar errores y probar cada parte por separado.

En definitiva, Haskell no es el lenguaje más sencillo para arrancar, pero te enseña a programar mejor sin importar con qué lenguaje termines trabajando después.

Bibliografía

<http://cufp.org/2011/fourteen-days-haskell-real-time-programming-projec.html>

<https://github.com/erkmos/haskell-companies>

https://wiki.haskell.org/Haskell_in_industry

<http://cufp.galois.com/2009/abstracts.html#ChristopherPiroEugeneLetuchy>

[https://www.process-one.net/content/files/2025/02/PiroLetuchy.pdf?](https://www.process-one.net/content/files/2025/02/PiroLetuchy.pdf)

<https://k1024.org/papers/icfp10-haskell-reagent.pdf>

<https://www.schoolofhaskell.com/>

<https://www.haskell.org/documentation/>